Exercise Sheet 10

Pivot Season

House rules.

- First: `pip install -r requirements.txt` inside the lesson folder (venv active).
- One file per exercise in `my_work/`, run from the lesson folder: `python my_work/ex1_headcount.py`.
- Every exercise starts with the **opener** (type it, don't paste it). Exercises 3, 4 and 6 also need the `hourly` column, **unrounded**:

```
import pandas as pd

pd.set_option("display.width", 120)
df = pd.read_csv("data/brightwell_payroll.csv")
df["hourly"] = df["annual_salary"] / 52 / df["weekly_hours"]
```

- Printed tables must match exactly, including the row order and the little `Name:` / `dtype:` lines pandas adds under a Series.
- Hand in: commit and push to `lesson-10`, then open a pull request. Tell Lars which PivotTable you'll never build by hand again.

Exercise 1 Headcount

5 min · `ex1_headcount.py`

> **From Siobhan (Reward):** *"Headcount by department, by site, and the gender split as a percentage. It's a COUNTIF, I know, but I have eleven of them and one is wrong."*

(a) Print `df["department"].value_counts()`. Empty line. The same for location. Empty line.

(b) The gender split as shares: `value_counts(normalize=True)`, rounded to 3 dp. Empty line.

(c) How many different job titles there are, and the three most common.

EXPECTED OUTPUT

```
department
Production    60
Sales         35
Warehouse     25
IT            20
Finance       18
Marketing     15
Design        15
HR            12
Name: count, dtype: int64

location
Manchester    93
Leeds         67
London        40
Name: count, dtype: int64

gender
M    0.585
F    0.415
Name: proportion, dtype: float64
```

```
Job titles: 6
job_title
Associate     43
Specialist    39
Assistant     38
Name: count, dtype: int64
```

Hint: `value_counts()` is a PivotTable with one row field and "Count" in the values. Sorted biggest first, always.

Exercise 2 The PivotTable

10 min · `ex2_pivot_by_department.py`

> **From Gerald (MD):** *"Average salary by department. And the count, median and max while you're there. And which department costs the most. One table, please, not four."*

After the opener: `by_dept = df.groupby("department")["annual_salary"]`.

(a) Print `by_dept.mean().round(0)`. Empty line.

(b) Print `by_dept.agg(["count", "mean", "median", "max"]).round(0)`. Empty line.

(c) Print the three biggest pay bills: `.sum()`, sorted largest first, `.head(3)`. Empty line.

(d) Print which department has the biggest pay bill and which has the highest average (`.idxmax()`).

EXPECTED OUTPUT

```
department
Design        33017.0
Finance       32547.0
HR            33129.0
IT            40415.0
Marketing     30910.0
Production    29148.0
Sales         32264.0
Warehouse     30844.0
Name: annual_salary, dtype: float64

            count     mean   median    max
department
Design         15  33017.0  31300.0  50150
Finance        18  32547.0  29275.0  68750
HR             12  33129.0  31250.0  63650
IT             20  40415.0  39925.0  60800
Marketing      15  30910.0  29000.0  51600
Production     60  29148.0  28650.0  50500
Sales          35  32264.0  28200.0  61750
Warehouse      25  30844.0  28950.0  47450

department
Production    1748900
Sales         1129250
IT             808300
Name: annual_salary, dtype: int64

Biggest pay bill: Production
Highest average: IT
```

Hint: `groupby("x")["y"]` says "row field x, values y". What comes after it (`.mean()`, `.sum()`, `.agg([...])`) is the "Summarize values by" box.

Exercise 3 Two row fields

10 min · `ex3_two_keys.py`

> **From Priya (Payroll):** *"The gap by department. Mean hourly rate for women and men, side by side, and the gap in percent, worst first. Gerald's words: 'worst first'."*

Opener plus the unrounded `hourly` column.

(a) Print the mean hourly rate by gender, 2 dp. Empty line.

(b) `both = df.groupby(["department", "gender"])["hourly"].mean()`. Print the first six rows, rounded to 2 dp. Empty line.

(c) `table = both.unstack().round(2)` turns the gender level into columns. Print it. Empty line.

(d) Add `gap_pct` = (M − F) / M × 100, rounded to 1 dp. Print the table sorted by gap, largest first.

EXPECTED OUTPUT

```
gender
F    14.95
M    18.62
Name: hourly, dtype: float64

department  gender
Design      F         17.23
            M         18.91
Finance     F         16.72
            M         19.81
HR          F         14.17
            M         25.81
Name: hourly, dtype: float64

gender          F      M
department
Design      17.23  18.91
Finance     16.72  19.81
HR          14.17  25.81
IT          17.05  23.86
Marketing   15.16  19.10
Production  14.03  15.88
Sales       14.52  19.05
Warehouse   11.56  18.07

gender          F      M  gap_pct
department
HR          14.17  25.81     45.1
Warehouse   11.56  18.07     36.0
IT          17.05  23.86     28.5
Sales       14.52  19.05     23.8
Marketing   15.16  19.10     20.6
Finance     16.72  19.81     15.6
Production  14.03  15.88     11.6
Design      17.23  18.91      8.9
```

Hint: Two keys need a **list**: `groupby(["a", "b"])`. And yes, that's HR at the top. Awkward.

Exercise 4 pivot_table, the real one

10 min · `ex4_pivot_table.py`

> **From Gerald:** *"Can it look like an actual PivotTable? Departments down, gender across, grand totals? I'm old."*

Opener plus the unrounded `hourly` column.

(a) `df.pivot_table(index="department", columns="gender", values="hourly", aggfunc="mean", margins=True)`, rounded to 2 dp. Empty line.

(b) A headcount table: same shape, `values="employee_id"`, `aggfunc="count"`, `fill_value=0`. Empty line.

(c) By site: `index="location"`, `aggfunc=["mean", "count"]` on hourly, rounded to 2 dp.

EXPECTED OUTPUT

```
gender          F      M    All
department
Design      17.23  18.91  18.01
Finance     16.72  19.81  18.26
HR          14.17  25.81  18.05
IT          17.05  23.86  21.48
Marketing   15.16  19.10  16.21
Production  14.03  15.88  15.26
Sales       14.52  19.05  17.24
Warehouse   11.56  18.07  16.51
All         14.95  18.62  17.10

gender       F   M
department
Design       8   7
Finance      9   9
HR           8   4
IT           7  13
Marketing   11   4
Production  20  40
Sales       14  21
Warehouse    6  19

             mean        count
gender          F      M     F   M
location
Leeds       15.42  19.12    26  41
London      13.98  17.76    20  20
Manchester  15.15  18.57    37  56
```

Hint: `margins=True` is the "Grand Total" row and column. `index` = Rows box, `columns` = Columns box, `values` + `aggfunc` = Values box.

Exercise 5 XLOOKUP

10 min · `ex5_xlookup.py`

> **From Mo (IT):** *"Finance keeps a separate list: department, cost centre, site manager, budget. It's in `data/departments.csv`. Priya XLOOKUPs it into the payroll every month. It's 200 XLOOKUPs. One of the departments is missing from Finance's list, as usual."*

Opener, then `depts = pd.read_csv("data/departments.csv")`.

(a) Print `depts`. Empty line.

(b) `merged = df.merge(depts, on="department", how="left")`. Print its shape and the first three rows of first_name, department, cost_centre, site_manager. Empty line.

(c) How many people have no cost centre (`.isna().sum()`)? Which department are they in (`sorted(... .unique())`)? Fill the gaps with `"CC-999"` using `.fillna()` and print the count again. Empty line.

(d) Budget check: pay bill per department (`groupby ... .sum().reset_index()`), merged with just `depts[["department", "budget"]]` using `how="inner"`. Add `headroom` = budget − pay bill and print sorted by headroom, worst first.

EXPECTED OUTPUT

```
   department cost_centre   site_manager   budget
0  Production      CC-100     Owen Jones  1750000
1   Warehouse      CC-110     Will Smith   700000
2       Sales      CC-200   Tom Robinson  1150000
3     Finance      CC-300  Lucy Thompson   540000
4          HR      CC-310   Harry Walker   400000
5          IT      CC-400   Nathan Smith   780000
6   Marketing      CC-500    Sam O'Brien   500000

(200, 15)
  first_name department cost_centre  site_manager
0        Max      Sales      CC-200  Tom Robinson
1     Imogen  Marketing      CC-500   Sam O'Brien
2      Amara      Sales      CC-200  Tom Robinson

15 people without a cost centre
['Design']
0 after fillna

   department  annual_salary   budget  headroom
6   Warehouse         771100   700000    -71100
0     Finance         585850   540000    -45850
2          IT         808300   780000    -28300
4  Production        1748900  1750000      1100
1          HR         397550   400000      2450
5       Sales        1129250  1150000     20750
3   Marketing         463650   500000     36350
```

Hint: `how="left"` keeps everyone from the left table, NaN where there's no match (XLOOKUP's "#N/A"). `how="inner"` keeps only matches, so Design silently disappears. Count your rows after every merge.

Exercise 6 Pay quartiles

10 min · `ex6_quartiles.py`

> **From Gerald:** *"The quartiles again, but as a proper table this time: four rows, women and men, counts and percentages. Last time it was five lines of text and the board asked for 'a grid'."*

Opener plus the unrounded `hourly` column, then `labels = ["Lower", "Lower middle", "Upper middle", "Upper"]`.

(a) `df["quartile"] = pd.qcut(df["hourly"], 4, labels=labels)`. Print `value_counts().sort_index()`. Empty line.

(b) `pd.crosstab(df["quartile"], df["gender"])`: counts. Empty line.

(c) The same with `normalize="index"`, times 100, rounded to 1 dp: the percentage of each quartile that is women / men. Empty line.

(d) The hourly range of each quartile: `groupby("quartile", observed=True)["hourly"].agg(["min", "max"])`, 2 dp.

EXPECTED OUTPUT

```
quartile
Lower           50
Lower middle    50
Upper middle    50
```

```
Upper           50
Name: count, dtype: int64

gender         F   M
quartile
Lower         36  14
Lower middle  23  27
Upper middle  15  35
Upper          9  41

gender           F     M
quartile
Lower         72.0  28.0
Lower middle  46.0  54.0
Upper middle  30.0  70.0
Upper         18.0  82.0

                min    max
quartile
Lower         10.26  13.21
Lower middle  13.23  15.38
Upper middle  15.46  19.83
Upper         19.95  35.26
```

Hint: `qcut` cuts into equal-sized groups (50 people each); `cut` would cut into equal-width ranges. The pay gap report wants `qcut`. Compare with lesson 08: same 72 % and 18 %.

Exercise 7 The bonus gap STRETCH

15 min · `ex7_bonus.py`

> **From Siobhan:** *"The report also wants the bonus gap: what share of women and men got a bonus, and the gap in the bonus amounts among those who did. I did it once in 2022. It took a day. The file is gone."*

Opener, then `df["got_bonus"] = df["bonus"] > 0`.

(a) Share receiving a bonus by gender: `groupby("gender")["got_bonus"].mean() * 100`, 1 dp. Empty line.

(b) `paid = df[df["got_bonus"]]`. Mean and median bonus by gender (`.agg(["mean", "median"])`), 2 dp. Empty line.

(c) Mean and median bonus gap, as a percentage of men's figure, 1 dp. Pick single cells out of a table with `table.loc["M", "mean"]`. Empty line.

(d) A pivot of mean bonus by department and gender (from `paid`), with a `gap_pct` column, rounded to 1 dp and sorted largest gap first.

EXPECTED OUTPUT

```
gender
F    45.8
M    66.7
Name: got_bonus, dtype: float64

            mean  median
gender
F       1614.21  1235.0
M       2090.13  1785.0

Mean bonus gap: 22.8%
Median bonus gap: 30.8%
```

```
gender           F       M  gap_pct
department
Warehouse    623.3  2573.6     75.8
IT          1192.0  3053.8     61.0
HR           780.0  1483.3     47.4
Design      1936.7  2135.0      9.3
Sales       1874.0  1791.3     -4.6
Production  2033.3  1865.8     -9.0
Marketing   1595.0  1350.0    -18.1
Finance     2437.5  1807.5    -34.9
```

Hint: A negative gap means women's average bonus was higher in that department. Four of eight. Gerald will ask why Warehouse is 75.8 %. (Nineteen men, six women, and the warehouse manager's bonus.)

Bug hunt Dave's Leftovers #10

15 min · `bughunt/gap_by_dept_bug.py`

Recovered from Dave's locked "Dave working" tab, where it was pasted into a cell as text. It's the gap by department with cost centres, for the board pack. **Three** bugs: one crashes (the error mentions a "level name"; think about how groupby takes two keys), one quietly loses fifteen people, one gets every percentage slightly wrong (compare with Exercise 3). One commit per bug, each message explaining what was wrong and why.

EXPECTED OUTPUT ONCE FIXED

```
People in report: 200
gender           F      M  gap_pct cost_centre
department
HR           14.17  25.81     45.1      CC-310
Warehouse    11.56  18.07     36.0      CC-110
IT           17.05  23.86     28.5      CC-400
Sales        14.52  19.05     23.8      CC-200
Marketing    15.16  19.10     20.6      CC-500
Finance      16.72  19.81     15.6      CC-300
Production   14.03  15.88     11.6      CC-100
Design       17.23  18.91      8.9         NaN
```